CI-BENCH: Evaluating LLM-Based Program Repair Tools on CI Failures

Raian Latif Nabil, Hao-Nan Zhu, and Cindy Rubio-González
University of California, Davis
Davis, California, USA
{mnabil, hnzhu, crubio}@ucdavis.edu

***Abstract*—Large language models (LLMs) have demonstrated their potential in performing complex software engineering (SE) tasks. Rigorous evaluation of LLMs and LLM-based tools requires massive, up-to-date data derived from real-world SE processes. Existing continuous integration & delivery (CI/CD) datasets, such as BUGSWARM, provide evolving data mined from software build processes, including complete context of build failures. To harness the CI/CD data, we propose CI-BENCH, a unified benchmarking framework designed to evaluate LLM-based program repair tools on software failures from CI/CD processes. CI-BENCH retrieves data from the BUGSWARM dataset, parses the build logs, and constructs appropriate prompts before invoking LLM-based program repair tools. Additionally, CI-BENCH includes an executor that facilitates dynamic evaluation in the identical environment as the original build process. With CI-BENCH, we evaluate three state-of-the-art LLM-based program repair tools, AGENTLESS, SWE-AGENT, and AUTOCODEROVER, on a code repair task involving 100 real-world CI/CD failures using GPT-4o, Claude-3.5-Sonnet, and Deepseek-V3 as foundation models. The evaluation shows that AGENTLESS, SWE-AGENT, and AUTOCODEROVER achieve success rates up to 27%, 30%, and 13% in generating correct patches, respectively. CI-BENCH is available at https://github.com/bugswarm/ci-bench and a demo of the tool can be found at https://youtu.be/y8-OsPCvDwY.**

***Index Terms*—Large Language Models, Benchmarking**

I. INTRODUCTION

Large Language Models (LLMs) have revolutionized the field of software engineering (SE) by enabling the automation of complex tasks such as fault detection, test generation, and automated code repair [16]. The rise of LLMs and LLM-based tools has transformed how researchers and developers approach SE tasks, allowing them to tackle challenges that traditional models often struggle with [8]. These tools are designed in various fashions, ranging from fixed, pre-defined steps [13] to agentic behaviors involving multi-step reasoning and planning [11].

However, evaluating the performance of LLMs and LLM-based tools on SE tasks remains a significant challenge. Existing benchmarks like SWE-Bench [9] and its extensions [10, 15] primarily focus on GitHub issues, which do not fully capture the complexities of real-world software development processes. Additionally, these benchmarks do not automatically evolve to include new data, leading to potential data leakage when evaluating LLMs with a later cutoff date than the benchmark release. Furthermore, these benchmarks lack sufficient details to fully replicate the original software failure, raising concerns about reproducibility.

We introduce CI-BENCH, a unified framework that enables the evaluation of various LLM-based program repair tools to address the limitations of existing benchmarks. To harness the rich context from the software build process, CI-BENCH retrieves software build failures from the BUGSWARM [12, 18] dataset, which is a collection of more than 3,500 CI/CD build failures containing complete context, such as execution environment, error messages, exceptions, and tracebacks. Such data is crucial for evaluating LLM-based program repair tools in real-world development and deployment scenarios. Furthermore, CI-BENCH also leverages the evolving nature of BUGSWARM to ensure that the evaluation remains up-to-date and based on the latest data available after the LLM's cutoff date. Lastly, CI-BENCH reconstructs an identical execution environment for each software build failure, ensuring that the evaluation is based on the faithful reproduction of real-world scenarios that developers face.

Given a targeted LLM-based program repair tool and a CI/CD build failure from the BUGSWARM dataset, CI-BENCH parses the build logs to extract information related to the failure. This information is then used to construct appropriate prompts, which serve as the input for the targeted tool. Additionally, using the environmental information from BUGSWARM, CI-BENCH sets up a containerized environment that mirrors the original build process, allowing the tool under evaluation to run in the identical environment as the original build failure. With this setup, CI-BENCH replays the CI/CD build process and automatically determines whether the targeted tool is successful in fixing the failure. To demonstrate the utility of CI-BENCH, we integrate three state-of-the-art LLM-based program repair tools: AGENTLESS [13], SWE-AGENT [14], and AUTOCODEROVER [17], with various underlying foundation models. However, CI-BENCH is not limited to these tools; it can be extended to support any LLM-based program repair tool that can process build failures.

We conduct an evaluation of the three LLM-based program repair tools on 100 real-world Java build failures from 59 Java projects, employing plausibility and syntactic equivalence as evaluation metrics. For each program repair tool, we assess the performance of different foundation models, including GPT-4o, Claude-3.5-Sonnet, and Deepseek-V3. Our results show that SWE-AGENT and AGENTLESS produce more correct

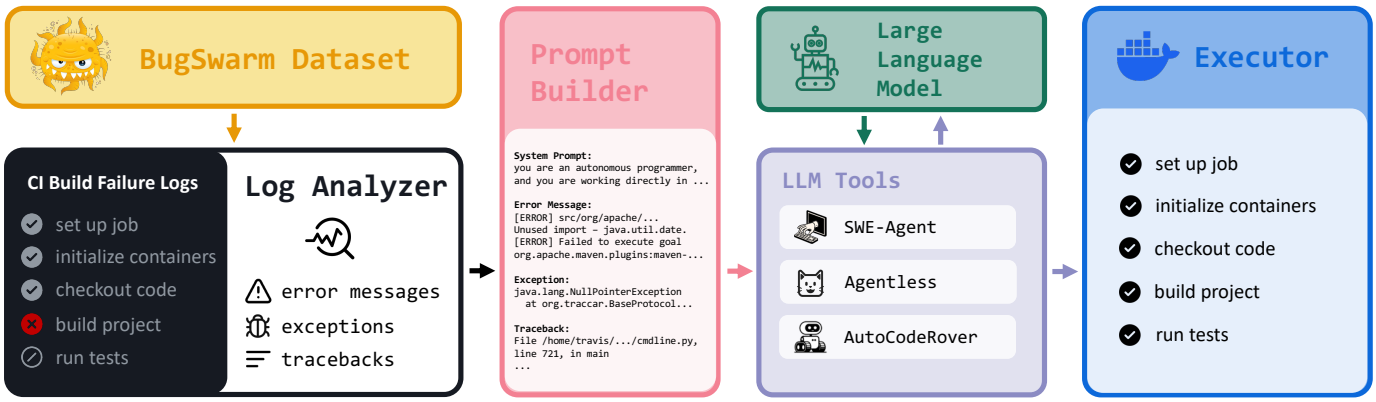

Fig. 1: Components of CI-BENCH

patches compared to AUTOCODEROVER in all models. SWE-AGENT and AGENTLESS both yield on average 26% to 28% of correctness with 100 artifacts we selected, outperforming AUTOCODEROVER.

With CI-BENCH, researchers and developers can easily evaluate LLM-based program repair tools using any subset of CI/CD build failures from the BUGSWARM dataset and obtain performance metrics. Its extensible design allows users to integrate new tools, models, and performance metrics seamlessly. The remainder of this paper presents the technical details of the framework and our evaluation results for AGENTLESS, SWE-AGENT, and AUTOCODEROVER with CI-BENCH.

II. OVERVIEW OF CI-BENCH

CI-BENCH is structured into several key components as shown in Figure 1. Our framework leverages the existing CI dataset BUGSWARM [12], as it is the largest and most diverse dataset of reproducible CI failures to date. The *Log Analyzer* extracts stack traces and tracebacks from historical CI failure logs provided by BUGSWARM. The *Prompt Builder* then constructs a generic prompt using this extracted information and feeds it to the LLM-based program repair tools. The tools integrate the prompt into their system prompt and interact with the LLM to generate a candidate patch. Finally, the *Executor* evaluates the correctness of the generated patch by applying it to the failed program, which is then tested using its original execution environment, faciliated by BUGSWARM via Docker and required build scripts. We further describe each component along with a running example using the BUGSWARM artifact `tananaev-traccar-64783123`.

A. The BugSwarm Dataset

CI-BENCH enables LLM-based program repair tools to be evaluated on existing CI datasets. In this paper, we select BUGSWARM, which is an established and maintained dataset built by mining consecutive failed-pass job pairs from two popular CI/CD services, Travis-CI [7] and GitHub Actions [3]. Each BUGSWARM artifact consists of a Docker image that includes the source code to reproduce the failure and the corresponding fix, scripts to build the code, and historical build logs from the original CI runs. The BUGSWARM dataset supports both Java and Python. Currently, it has 3,598 active artifacts; 2,245 are Java artifacts, and the rest are Python.

In the rest of this section we will consider the BUGSWARM artifact `tananaev-traccar-64783123`. This artifact is mined from a Java GitHub repository named `traccar` [6], where a Null Pointer Exception occurs. We use this artifact to showcase our evaluation of the SWE-AGENT tool.

B. Log Analyzer

CI-BENCH's first component is the *Log Analyzer*, which parses the failed build log of the artifact to extract error messages, exceptions, and tracebacks. This information is important because it usually includes the location of the failure such as file and line number, class name and method name. For our example, artifact `tananaev-traccar-64783123`, the Log Analyzer processes the failed build log and extracts various error messages, the Null Pointer Exception, and its tracebacks that indicate the location of the failure. This information serves as input to the *Prompt Builder*.

C. Prompt Builder

The *Prompt Builder* creates a generic prompt from the error messages, exceptions, and/or tracebacks provided by the Log Analyzer to guide LLMs in code repair tasks. This generic prompt is the same for all the considered tools and defines their objective when processing the given failure information. Each tool has its system prompt, a predefined instruction that sets the overall task and its behavior. The generic prompt is inserted within this system prompt to specify the task. The tools are originally designed to fix bugs from bug reports, not from build logs, thus the need for our own *Prompt Builder*.

D. Integrated LLM-Based Program Repair Tools

CI-BENCH is a unified framework that can plug in specific LLM program repair tools for benchmarking. To use the CI-BENCH framework, tools must meet the following requirements: (1) provide support for Java code repair, (2) be capable of processing sequential text input (required to process

```
protected Object decode( /* ... */ ) {
  /* ... */
  } else if (type == MSG_LOGIN) {
-   if (channel == null) {
+   if (channel != null) {
      /* ... */
      response.writeByte(0x0A);
      channel.write(response, remoteAddress);
    /* ... */
    }
  /* ... */
```

(a) Patch made by the developer.

```
protected Object decode( /* ... */ ) {
  /* ... */
  } else if (type == MSG_LOGIN) {
    if (channel == null) {
      /* ... */
      response.writeByte(0x0A);
-     channel.write(response, remoteAddress);
+     if (channel != null) {
+       channel.write(response, remoteAddress);
+     }
    /* ... */
```

(b) Patch generated by SWE-AGENT.

Fig. 2: Patches for `tananaev-traccar-64783123`.

CI/CD build logs), and (3) generate patches. Currently, CI-BENCH has integrated three state-of-the-art LLM-based program repair tools, AGENTLESS [13], SWE-AGENT [14], and AUTOCODEROVER [17]. While these tools were originally designed for Python, we adapted them for Java programs. We also modified the tools to interact with BUGSWARM containers instead of relying on GitHub issue threads or source repositories. For example, we adapted the parsing and filtering logic to handle Java files and functions, and modified prompt templates to reflect Java syntax.

E. Executor

The *Executor* validates the tool's generated patches. It takes the patch file as input and applies it within the artifact's Docker container that includes all necessary dependencies for building. The Executor runs all the commands the original failed job tried to run. If all commands are executed and the repository is built successfully after applying the patch, and there is no failed tests and error, then the patch is considered as a correct patch.

For the example artifact, the patch generated by SWE-AGENT, shown in Figure 2b, results in a successful build and is therefore considered correct. However, the generated patch is not syntactically equivalent to the patch made by the developer, shown in Figure 2a. We discuss the performance of the tools using different metrics in Section III.

III. EVALUATION

We aim to determine how the LLM-based code repair tools perform on the CI failure data. We vary the foundation models within the LLM-based code repair tools and assess the performance of each tool using different correctness metrics.

a) Artifact and Model Selection: We adopt SWE-Bench Lite [4]'s issue selection criteria to select BUGSWARM artifacts for this evaluation: (1) patches that do not include images, external hyperlinks, or references to specific commit SHAs and references to other pull requests or issues; (2) patches that contain one file edit; (3) patches with three or fewer edit hunks; (4) patches that do not create or remove files; (5) patches that are not for test files. From those artifacts, we selected 100 from 59 projects. We chose three foundational models that have recently performed better on code generation tasks: GPT-4o, Claude-3.5-Sonnet, and Deepseek-V3.

b) Correctness Metrics: We use two metrics to measure correctness: plausibility (PL) and syntactic equivalence (SYE). Plausibility is the measure of correct patches that result in a successful build of a BUGSWARM artifact. Syntactic equivalence refers to the token-level equivalence between the generated patch and the developer patch. CI-BENCH uses ANTLR [2] to tokenize the patches before comparing the generated patch with the developer patch.

c) Evaluation Results: As shown in Table I, we observe that all three tools are able to generate patches with all three functional models. AGENTLESS and SWE-AGENT generate more patches than AUTOCODEROVER in general, with up to 52 and 51 patches generated, respectively. Our investigation shows that AUTOCODEROVER generates fewer patches due to the non-deterministic behavior of its patch agent, which sometimes fails to locate the exact code region for applying the patch, even when the patch is successfully generated by the functional model. We have also investigated the reasons why AGENTLESS and SWE-AGENT do not generate patches for the remaining artifacts. For AGENTLESS, the main reason is that it fails to localize the bug location correctly, which is an essential step before generating a patch. And SWE-AGENT fails to generate patches due to the cost limit of the functional model, which prevents it from generating a patch for every artifact.

Not all generated patches are plausible (PL) to make a successful build. Among the generated patches, AGENTLESS achieves up to 27 plausible patches across the three models, while SWE-AGENT achieves a comparable number (up to 30), indicating their ability to effectively fix the issues that block the build process. On the other hand, AUTOCODEROVER achieves up to 13 plausible patches, which is lower than the other two tools.

When considering syntactic equivalence (SYE), which indicates patches that are not only plausible but also syntactically equivalent to the ground truth, the trend persists. AGENTLESS and SWE-AGENT consistently outperform AUTOCODEROVER, achieving up to 22 syntactically equivalent patches across the models. Notably, Claude-3.5-Sonnet leads

TABLE I: Comparison of different models across tools on 100 artifacts. (#PG: Patches Generated, #PL: Plausible Patches, #SYE: Syntactic Equivalence)

	AGENTLESS			SWE-AGENT			AUTOCODEROVER		
Model	#PG	#PL	#SYE	#PG	#PL	#SYE	#PG	#PL	#SYE
GPT-4o	51	27	20	51	29	22	21	13	15
Claude-3.5-Sonnet	52	26	22	44	26	22	24	10	12
Deepseek-v3	34	21	18	47	30	27	5	4	4

with 22 SYE patches under AGENTLESS, suggesting stronger alignment with the human-made fixes. The performance of Deepseek-V3 is quite notable under SWE-AGENT is due to its lower API cost, which enables a higher frequency of agent interactions within a fixed budget.

Comparing the foundation models, GPT-4o and Claude-3.5-Sonnet perform similarly across all tools, with Claude-3.5-Sonnet slightly ahead when considering syntactic equivalence. However, Deepseek-V3 generates fewer patches that are plausible or syntactically equivalent with AUTOCODEROVER, indicating a potential limitation in its code repair capabilities compared to the other two models.

IV. RELATED WORK

SWE-Bench [9] is a benchmark of 2,294 real-world GitHub issues from which the task of code repair is performed. SWE-Bench Lite [4] is a subset of SWE-Bench, popular among SE research communities. SWE-Bench Verified [5] is another subset of SWE-Bench with human annotation released by OPENAI. Other benchmarks such as Multi-SWE-Bench [15] and Poly-SWE-Bench [10] expand the benchmark to support multiple programming languages beyond Python. Coding agents like SWE-AGENT [14], AUTOCODEROVER [17], AIDER [1] have shown significant performance on these datasets. Non-agent tools like AGENTLESS have also shown better performance. However, a key limitation of these benchmarks is that they lack dataset diversity and do not include reproducibility studies. Additionally, these benchmarks are centered around GitHub issues. In contrast, BUGSWARM provides reproducible CI failure data for both Java and Python, covering a broader range of repositories. CI-BENCH is a unified framework that leverages LLM tools that work on CI/CD data, enabling systematic evaluation of LLM-based tools. This differentiates CI-BENCH from prior works by benchmarking a more diverse and reproducible dataset to evaluate LLM tools.

V. CONCLUSION

In this paper, we presented CI-BENCH, the first benchmark to evaluate LLM-based program repair tools on CI failures. Existing benchmarks lack reproducibility and recency and we overcome it by leveraging BUGSWARM. Our CI-BENCH framework integrates three state-of-the-art LLM-based program repair tools. With CI-BENCH, we evaluated these tools on 100 real-world artifacts from BUGSWARM. The evaluation shows that AGENTLESS and SWE-AGENT outperform AUTOCODEROVER in generating correct patches. In the future, we plan to extend our framework to evaluate other tasks such as test generation, fault localization, bug report generation, and commit message generation.

REFERENCES

[1] AIDER. https://aider.chat/2024/06/02/main-swe-bench.html, 2025.

[2] antlr. https://www.antlr.org/, 2025.

[3] GitHub Actions. https://docs.github.com/en/actions/, 2024.

[4] SWE-bench Lite. https://www.swebench.com/lite.html, 2025.

[5] SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/, 2025.

[6] traccar. https://github.com/traccar/traccar/, 2025.

[7] Travis-CI. https://travis-ci.org/, 2024.

[8] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang. Large language models for software engineering: A systematic literature review. *ACM Trans. Softw. Eng. Methodol.*, 33(8):220:1–220:79, 2024.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[10] M. S. Rashid, C. Bock, Y. Zhuang, A. Buchholz, T. B. Esler, S. Valentin, L. Franceschi, M. Wistuba, P. T. Sivaprasad, W. J. Kim, A. Deoras, G. Zappella, and L. Callot. Swe-polybench: A multi-language benchmark for repository level evaluation of coding agents. *CoRR*, abs/2504.08703, 2025.

[11] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths. Cognitive architectures for language agents. *Trans. Mach. Learn. Res.*, 2024, 2024. URL https://openreview.net/forum?id=1i6ZCvflQJ.

[12] D. A. Tomassi, N. Dmeiri, Y. Wang, A. Bhowmick, Y. Liu, P. T. Devanbu, B. Vasilescu, and C. Rubio-González. Bugswarm: mining and continuously growing a dataset of reproducible failures and fixes. In *ICSE*, pages 339–349. IEEE / ACM, 2019.

[13] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying llm-based software engineering agents. *CoRR*, abs/2407.01489, 2024.

[14] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

[15] D. Zan, Z. Huang, W. Liu, H. Chen, L. Zhang, S. Xin, L. Chen, Q. Liu, X. Zhong, A. Li, S. Liu, Y. Xiao, L. Chen, Y. Zhang, J. Su, T. Liu, R. Long, K. Shen, and L. Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving. *CoRR*, abs/2504.02605, 2025.

[16] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang, W. Sun, S. Yu, and Z. Chen. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223*, 2023.

[17] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury. Autocoderover: Autonomous program improvement. In *ISSTA*, pages 1592–1604. ACM, 2024.

[18] H. Zhu, K. Z. Guan, R. M. Furth, and C. Rubio-González. Actionsremaker: Reproducing GITHUB actions. In *ICSE Companion*, pages 11–15. IEEE, 2023.